

Multilevel Feedback Queue (MLFQ)

MLFQ gives new processes the highest priority. Processes that use their CPU allowance move to a lower priority. A periodic boost returns active processes to the top queue.

1. Queues and CPU budgets

The visualizer has one CPU and three ready queues: Q0 is highest priority, followed by Q1, then Q2. Time advances in whole ticks. Processes need only CPU service, with no I/O blocking or cost for switching processes.

Three quantities determine how long a process runs:

<i>Quantity</i>	<i>Meaning</i>
Remaining service	CPU ticks still needed to finish the process.
Quantum (time slice)	CPU ticks allowed in one turn at the current priority.
Allotment	Total CPU ticks allowed at the current priority before demotion.

Quantum and allotment are separate settings. Their defaults happen to be equal:

<i>Queue</i>	<i>Quantum</i>	<i>Allotment</i>
Q0	2	2
Q1	4	4
Q2	8	8

The default boost interval is 10 ticks. Every budget and the boost interval must be a positive whole number. The quantum need not divide the allotment or be smaller than it.

Each process has two usage counters. For example, **Q: 1/2** and **A: 3/4** mean that it has used one tick of its current two-tick turn and three ticks of its four-tick allowance at this priority. Its remaining service is a separate value.

A ready process has arrived and is waiting for the CPU. A future process has not arrived yet. Queue lists run from head to tail: in [B, C], B is first. The short examples under the rules are separate scenarios. Unless a boost is mentioned, assume none is due during that example.

2. Choosing a process

Rule 1. New arrivals join Q0

Append a new process to the tail of Q0 with both usage counters at zero. Simultaneous arrivals enter in input order. The leftmost process in a queue is its head.

Explanation. Every new process gets a first turn at the highest priority, regardless of its service time. Processes already waiting in Q0 keep their places ahead of it. A future process joins no ready queue until its arrival time.

Example. B is waiting in Q0. C and A arrive together, with C listed before A in the input. Before dispatch, Q0 is [B, C, A]. C and A both start with zero quantum and allotment usage. Their process names do not affect this order.

Rule 2. Choose the highest-priority ready queue

When the CPU is free, run the head of the highest-priority nonempty queue. Processes within a queue take turns in Round Robin order.

Explanation. First choose the priority level, then take that queue's head. A shorter service time does not let a process jump ahead within a queue or ahead of a higher-priority process.

Example. Q0 is empty, Q1 contains [B, C], and Q2 contains [D]. Select B. C waits for its turn in Q1. D cannot be selected from Q2 while either higher-priority queue has ready work.

Rule 3. Higher-priority work interrupts lower-priority work

If a higher-priority process becomes ready, preempt the lower-priority running process. Return the interrupted process to the **front** of its own queue and preserve both usage counters. Same-priority arrivals do not interrupt a turn.

Explanation. An ordinary priority preemption pauses a turn. It does not give the interrupted process a fresh turn or erase CPU time already charged to its allotment. Returning to the front lets it resume ahead of its peers when its priority is selected again.

Example. A is running in Q1 with quantum usage 1/4 and allotment usage 5/6 when B arrives in Q0. B preempts A, which returns to the front of Q1. When A resumes, it still has three quantum ticks but only one allotment tick available, unless a boost has reset its budgets. One more CPU tick will therefore exhaust its allotment before its quantum.

If A had been running in Q0 with budget remaining, B's arrival in Q0 would not interrupt it. B would wait at the tail of Q0.

3. Accounting for CPU time

Rule 4. Charge only the process that runs

For each tick of execution, reduce remaining service by one and increase both quantum usage and allotment usage by one. Waiting uses neither budget.

Explanation. All three counters measure CPU execution, not elapsed waiting time. Quantum usage counts the current turn; allotment usage also includes earlier turns at this priority. Remaining service counts the work still needed across every priority level.

Example. A runs one tick in a queue with quantum 4 and allotment 6:

<i>Counter</i>	<i>Before the tick</i>	<i>After the tick</i>
Remaining service	5	4
Quantum used	1/4	2/4
Allotment used	1/6	2/6

If B waits during this tick, B's remaining service and both usage counters stay unchanged.

Rule 5. Completion takes precedence

At the next tick boundary, remove a process whose remaining service has reached zero. It is not rotated, demoted, or boosted, even if any of those events would also occur at that boundary.

Explanation. Budget expiry and boosting only affect unfinished work. Checking completion first prevents a finished process from receiving new budgets or returning to a ready queue. Completion and the next dispatch take no extra CPU tick.

Example. Set Q0's quantum and allotment to 2 and the boost interval to 2. A starts at time 0 needing two service ticks; B is waiting. At time 2, A finishes exactly when both budgets expire and a boost is due. Remove A first. Boost B and dispatch it. A is absent from the boosted queue.

Rule 6. Exhausting the allotment ends the priority level

At a boundary without a boost, if an unfinished process has used its allotment, append it to the next lower queue and reset both usage counters. Thus Q0 moves to Q1, and Q1 moves to Q2. A process already in Q2 rejoins Q2's tail with fresh budgets.

Allotment exhaustion takes precedence over quantum expiry and can happen partway through a turn. If a boost is due at the same boundary, apply Rule 9 instead of demoting.

Explanation. The allotment limits total CPU use at a priority, across multiple turns. Starting another turn does not restore it. Demotion gives the process the next queue's budgets; at the lowest priority, exhausting the allotment instead renews that queue's budgets and sends the process to its tail.

Example. A is alone, needs ten service ticks, and starts in Q0 with quantum 2 and allotment 3. Its first turn runs from 0 to 2. Its second turn lasts only one tick:

<i>Boundary</i>	<i>Result</i>	<i>Quantum used</i>	<i>Allotment used</i>
Time 1	A continues in Q0	1/2	1/3
Time 2	A starts another Q0 turn	0/2	2/3
Time 3	A moves to Q1	0/4	0/4

Q1 uses its default budgets of 4. At time 3, A has seven service ticks left. Its Q0 allotment ended before the second turn's quantum could finish.

For the lowest-queue case, suppose A exhausts its Q2 allotment while B waits in Q2. Requeue A behind B, forming [B, A]. A stays in Q2 with both counters reset; there is no Q3.

Rule 7. Quantum expiry alone ends only the turn

At a boundary without a boost, if the quantum expires but allotment remains, append the unfinished process to the tail of the same queue. Reset quantum usage and preserve allotment usage.

Explanation. A quantum divides a priority's allotment into turns so that peers can share the CPU. Ending a turn does not necessarily lower priority. With quantum 1 and allotment 4, a process can take four one-tick turns at that level before demotion, provided it does not finish or receive a boost first.

Example. Q0 has quantum 2 and allotment 6. A starts at time 0 needing five service ticks, with B waiting in Q0. At time 2, A has three service ticks left and has used only two of its six allotment ticks. It rejoins Q0 behind B:

<i>State of A after requeueing</i>	<i>Value</i>
Priority	Q0
Remaining service	3 ticks
Quantum used	0/2
Allotment used	2/6

B runs next. If C also arrives at time 2, admit C before requeueing A, giving [B, C, A] before dispatch. A's allotment usage remains 2/6 in either case.

Rule 8. Keep the CPU busy when work is ready

After a turn ends, dispatch normally. If the process is alone, it may immediately run again, either at its existing priority or after demotion. No idle tick is inserted. Idle only when nothing is ready; stop when all processes finish.

Explanation. Moving a process between queues takes no CPU time. An empty waiting queue does not stop a process already running, and a future process cannot be dispatched early.

Example. A is alone, arrives at time 0, and needs five service ticks. Use the default budgets. A runs in Q0 from 0 to 2, then immediately continues in Q1 from 2 to 5 and finishes. The demotion at time 2 adds no idle gap.

If A had instead arrived at time 3, the CPU would be idle from 0 to 3. Only after A arrives can it begin its first Q0 turn.

4. Priority boosts

Rule 9. Boost waiting and running processes

A boost occurs at positive multiples of the boost interval, never at time 0. For interval 10, these are times 10, 20, 30, and so on.

At a boost:

1. Collect waiting processes from Q0, then Q1, then Q2. Preserve the order within each queue.
2. Stop the unfinished running process and append it after all the waiting processes. Do this even if its quantum or allotment has just expired.
3. Put the collected processes in Q0 and reset both usage counters. Preserve remaining service.
4. Admit any arrivals at this time, then dispatch from Q0.

Explanation. A boost gives waiting processes a turn at the highest priority. The unfinished running process joins behind them because it has just received CPU time. Completion is checked first; otherwise the boost replaces any same-boundary rotation or demotion. The running process moves directly from the CPU to the end of Q0.

This also applies to a process already running in Q0. If nobody else is waiting or arriving, the interrupted process is immediately selected again with fresh budgets.

Example 9a. A boost during an ongoing turn

A is running in Q1 without having exhausted either budget. Q0 is empty, B and C wait in Q1, and D waits in Q2.

<i>Stage</i>	<i>CPU</i>	<i>Q0</i>	<i>Q1</i>	<i>Q2</i>
Before boost	A	Empty	B, C	D
After boost, before dispatch	Free	B, C, D, A	Empty	Empty
After dispatch	B	C, D, A	Empty	Empty

All four processes have zero quantum and allotment usage immediately after the boost. A waits behind the other three; its unfinished service has not changed. If E also arrives at this boundary, it joins after A, giving [B, C, D, A, E] before dispatch.

Example 9b. Allotment expiry coincides with a boost

A exhausts its Q0 allotment but still needs service. B waits in Q0, C in Q1, and D in Q2.

<i>Stage</i>	<i>CPU</i>	<i>Q0</i>	<i>Q1</i>	<i>Q2</i>
Before handling the boundary	A	B	C	D
After boost, before dispatch	Free	B, C, D, A	Empty	Empty
After dispatch	B	C, D, A	Empty	Empty

Collect B, C, and D first, then append A. A does not pass through Q1. All four processes receive fresh Q0 budgets, and B runs next. A new arrival E would join after A, making [B, C, D, A, E] before dispatch.

The same order applies if only A's quantum expires. If A finishes its service at this boundary, it leaves first and the boost collects [B, C, D].

5. Events at the same boundary

The order below is part of the implemented policy. It matters when expiry, boost, and arrival happen together. In both cases, first account for execution during the preceding tick.

Without a boost

1. Remove the running process if it has finished.
2. Check for allotment exhaustion, then quantum expiry, in any unfinished running process.
3. Add all arrivals at the current time.
4. Requeue the process whose budget expired, applying Rule 6 or Rule 7.
5. If a process is still running, check for higher-priority preemption.
6. Dispatch if the CPU is free.

Arrivals therefore enter before expired work is requeued. An existing waiting process keeps its place within its queue.

With a boost

1. Remove the running process if it has finished.
2. Collect waiting processes from Q0, then Q1, then Q2, preserving queue order.
3. Append the unfinished running process last, even if a budget expired at this boundary.
4. Put everyone collected in Q0 and reset both usage counters. Preserve remaining service.
5. Add all arrivals at the current time.
6. Dispatch from Q0 if any process is ready.

Do not demote or rotate the running process before this boost. Completion comes first; the boost then takes precedence over either kind of budget expiry.

Example 9b follows the boost order even though A's allotment has expired. Rule 7's arrival-and-expiry example follows the ordinary order because no boost is due there.

6. Worked example: separate quantum and allotment

A and B both arrive at time 0, in that order, and each needs ten service ticks. Set Q0's quantum to 1 and allotment to 4. Leave Q1 and Q2 at their defaults and set the boost interval to 100, so there is no boost during this example.

The complete schedule is:

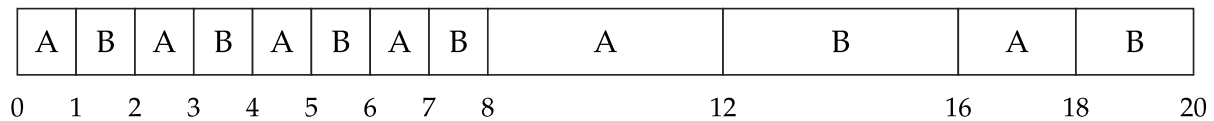


Figure 1. CPU schedule (time in ticks).

After A's first, second, and third turns, its quantum usage resets to zero while its allotment usage is 1, 2, and 3. At time 7, A has used four CPU ticks in Q0. It moves to Q1 with both counters reset and six service ticks left.

B is still in Q0, so B gets the CPU for the tick from 7 to 8. At time 8, B also moves to Q1 with six service ticks left. Q1 now contains [A, B] before dispatch, so A runs next.

A uses four ticks in Q1 from 8 to 12, then moves to Q2 with two service ticks left. B uses Q1 from 12 to 16 and also moves to Q2 with two left. Q2 contains [A, B] before dispatch at time 16. A finishes at time 18, and B finishes at time 20. Each received ten CPU ticks in total.